
Summarizing Data

Location, variability, and outliers

Prof. Gabriel Spadon, PhD

Faculty of Computer Science
spadon@dal.ca

Learning objectives

By the end of this class, students should be able to

- distinguish numeric from categorical variables, including ordinal categories, and say which summaries each type admits;
- calculate statistics of location: mean, median and mode;
- calculate statistics of variability: variance, standard deviation, interquartile range and range;
- read and construct quartiles, and state which sample-quantile definition they used;
- flag possible outliers using the boxplot rule; and
- generate different global dataset summaries in R and in Python, and explain how skewness and outliers distort them.

Class A introduced tables and the data science process. This class describes the distribution of a variable, meaning how its values are spread across observations. We calculate summaries by hand, check them in R or Python, and then apply them to a small CSV before beginning Lab 01. Optional research examples follow the required material.

1 Opening business

1.1 Announcements

- Prepare R or Python before Lab 01, using a local environment or a free cloud notebook.
- Work through the summary-statistics example during the class warm-up.
- Course-specific deadlines are published in the course GitHub repository.

1.2 Environment and access

Choose Posit Cloud or Kaggle Notebooks for R; for Python, use Posit Cloud, Kaggle Notebooks, or Google Colab. Local R with RStudio or local Python with JupyterLab or VS Code is equally acceptable. No paid service is required. Keep assessed cloud notebooks private until the deadline and the instructor opens sharing. Use Microsoft Teams for course questions and contact the instructor early if access or installation is a barrier.

2 Recap: process models and the shape of a data science project

2.1 Ungraded recap

Name the six CRISP-DM stages introduced in Class A. Where would you check a temperature column for missing readings? Which stage would you return to if the units needed correcting? Checking the measurements belongs to data understanding; correcting their representation belongs to data preparation. The cycle in Figure 1 gives the stage order.

2.2 What data science is

Data science uses computing, statistics, and domain knowledge to learn from data and answer questions. Large datasets are not required. It sits at the intersection of computer science and information technology, mathematics and statistics, and domain or business knowledge. The pairwise intersections have their own names: machine learning between computing and statistics, software development between computing and the domain, and traditional research between statistics and the domain. These labels are a simplified teaching diagram, not exclusive boundaries. Figure 1 shows how the disciplines contribute to the same analysis.

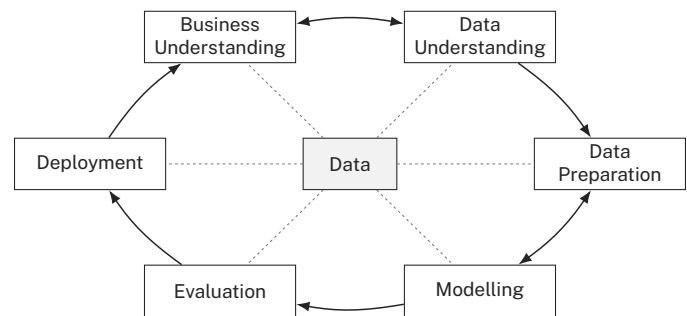
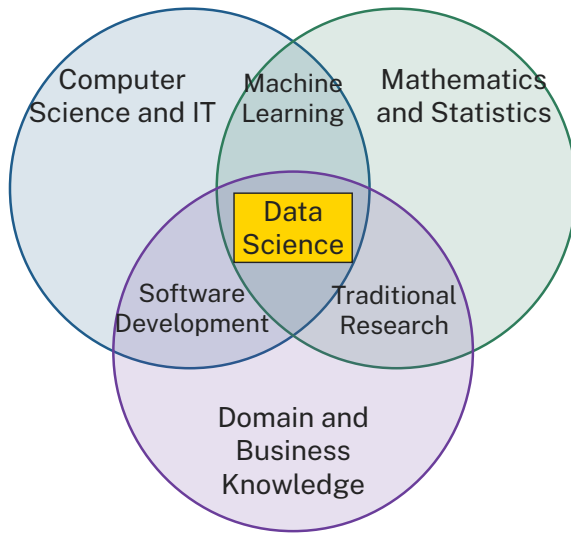


Figure 1. The same three-discipline intersection and six-stage CRISP-DM cycle introduced in Class A. The cycle includes iteration and feedback.

CRISP-DM is a popular *non-linear* workflow for data science projects. Business understanding and data understanding inform each other; data preparation and modelling iterate; evaluation can send the project back to the beginning before anything is deployed. Data science tasks in this course are categorized into exploratory data analysis, descriptive analytics, and predictive analytics.

2.3 Today's position on the course map

Figure 2 expands data understanding into importing and exploratory analysis. Summarizing and visualizing are sibling activities within exploratory analysis. Gold marks today's focus; a topic connection does not impose an algorithmic sequence.

Today's material is the highlighted box. The next classes develop *Data Preparation*, beginning with preprocessing foundations. Predictive and descriptive modelling follow later, in the order set out in the syllabus schedule. The modelling detail in Figure 3 is the same one used in Class A and in the corresponding recap slide.

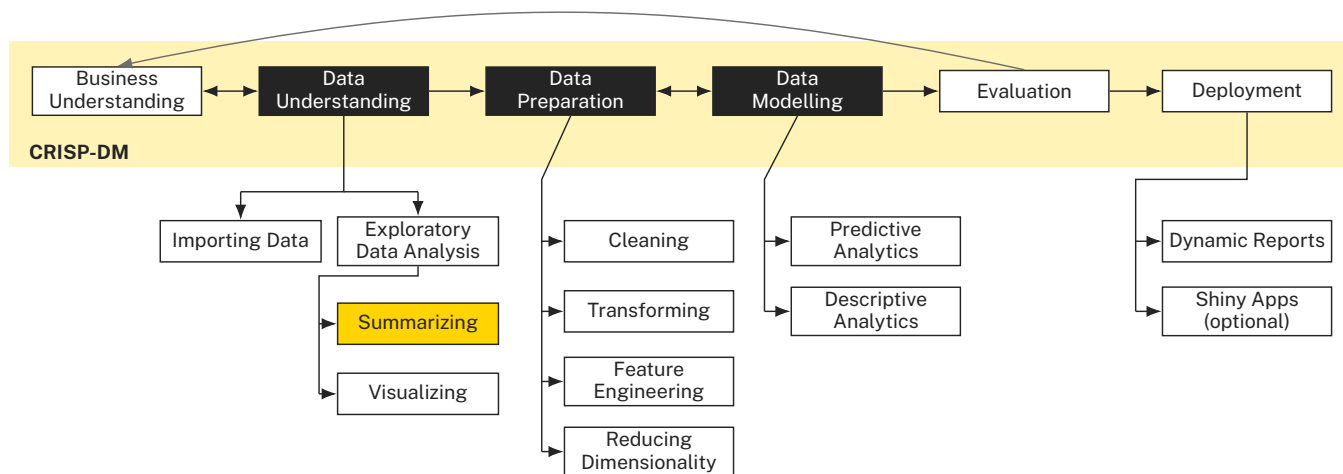


Figure 2. Course syllabus mapped onto the CRISP-DM stages

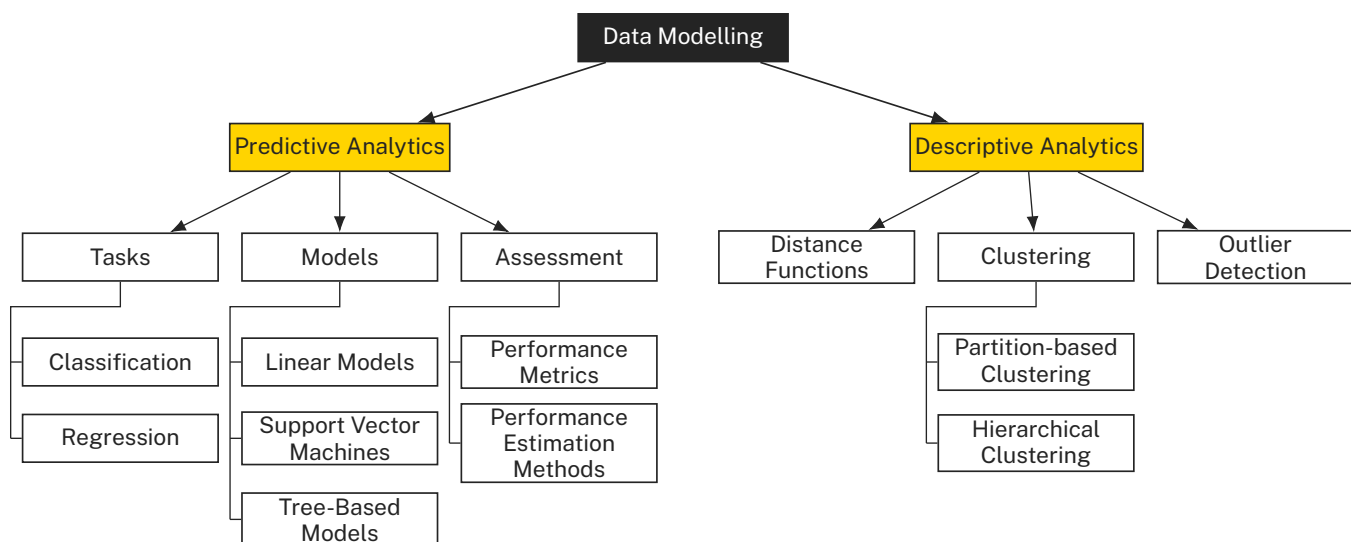


Figure 3. Predictive and descriptive modelling topics carried forward from Class A. Connections group methods; they do not require using each algorithm in sequence.

3 Why summarize data, and what kind of variable is it?

3.1 Why summarize

Data are often too large for us to be able to understand what is going on just by perusing them. Data summaries give us an overview of key data properties. We want to describe important properties of the *distribution* of the values across the measured observations. Reading every cell of a table with 2 431 rows and 18 columns is impractical. A summary gives an entry point, while plots and source-row inspection retain the context that a handful of numbers can hide.

3.2 Types of variables

Table 1 distinguishes quantities from labels. This distinction determines which calculations will be meaningful.

The variable type decides which summary is even defined. A mean of a nominal variable is not meaningful, no matter what the software returns when the categories happen to be stored as integers.

2 431 rows × 18 columns

a handful of numbers

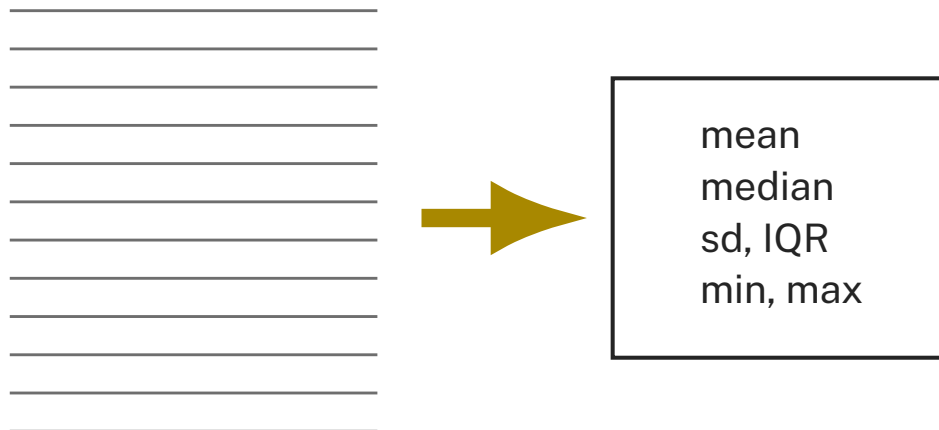


Figure 4. The same rows-to-summary diagram used in the slides. Summaries condense the table; they do not replace checking its distributions and source observations.

Table 1. Variable types and what their values represent

Type	Example	Meaning of the values
Numeric, discrete	number of clinic visits	counts taking separate possible values
Numeric, continuous	temperature in degrees Celsius	measurements on a continuous scale, recorded to finite precision
Nominal categorical	recorded blood group	labels without a numerical order
Ordinal categorical	low, moderate, high severity	ordered labels; the gaps between categories need not be equal

Storage types do not determine analytical roles. R stores the whole-number values below as doubles; they can still represent a discrete variable.

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)

class(x)
str(x)
```

Python

```
import pandas as pd
x = pd.Series([2, 4, 4, 5, 7, 9, 12, 41])

x.dtype
```

4 Three questions to ask of a variable

The three questions

- Q1.** What is the “typical or common value” of a variable?
- Q2.** What is the “variability” in the values of a variable?
- Q3.** Are there “strange” or unexpected values in the dataset?

Use the same eight values throughout the class, $x = 2, 4, 4, 5, 7, 9, 12, 41$. In R, `c(...)` collects values into a vector. In Python, `pd.Series([...])` creates a labelled column. These are practice values, not environmental observations.

5 Q1. Statistics of location

5.1 Mean, median, mode

The sample mean is

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

The **median** has at least half the observations at or below it and at least half at or above it, allowing ties. For numeric data, sort and pick the middle value; if there are an even number of values, calculate the mean of the two in the middle. The **mode** is the most frequently occurring value, and it is the only one of these three summaries that applies to unordered nominal categories.

Worked example

For $x = 2, 4, 4, 5, 7, 9, 12, 41$ with $n = 8$:

- mean: $\bar{x} = (2 + 4 + 4 + 5 + 7 + 9 + 12 + 41)/8 = 84/8 = 10.5$;
- median: the two middle values of the sorted vector are 5 and 7, so the median is $(5 + 7)/2 = 6$;
- mode: the value 4 occurs twice and every other value once, so the mode is 4.

The single value 41 pushes the mean to 10.5 while the median stays at 6.

R

```
mean(x)      # 10.5
median(x)    # 6

# ignore missing values
mean(c(2, NA, 4), na.rm = TRUE) # 3

# mode() reports storage, not frequency
tab <- table(x)
names(tab)[tab == max(tab)]      # "4", all modes
```

Python

```
x.mean()      # 10.5
x.median()    # 6.0

# pandas drops NaN by default
pd.Series([2, None, 4]).mean() # 3.0

x.mode().tolist() # [4]
```

A missing value is recorded as NA in R and as NaN in a numeric pandas column. R requires `na.rm = TRUE` to omit it; pandas omits it by default. Both examples average the two observed values, 2 and 4, and return 3. Report the observed count and the missing count separately. The R frequency table and pandas' mode return all tied modes.

5.2 Getting an accurate “picture” of your data

This makes the fragility of the mean concrete.

Ten people have an average income of \$50,000. Replace the highest earner with someone earning far more. The new average can be arbitrarily large, while the two middle incomes stay exactly the same.

Hypothetical income example.

Discussion prompt: what happens to the median in this example? It does not change. Replacing the largest of ten values with a larger value leaves the fifth and sixth values unchanged. The mean has no such protection. It is a weighted sum, so one unbounded value drags it without limit. This is the difference between a *robust* statistic and a *non-robust* one.

This illustrates the same effect with a histogram of the square footage of 25 homes, shown in Figure 5. Twenty-three homes lie between 1 400 and 2 800 square feet and two are near 7 000.

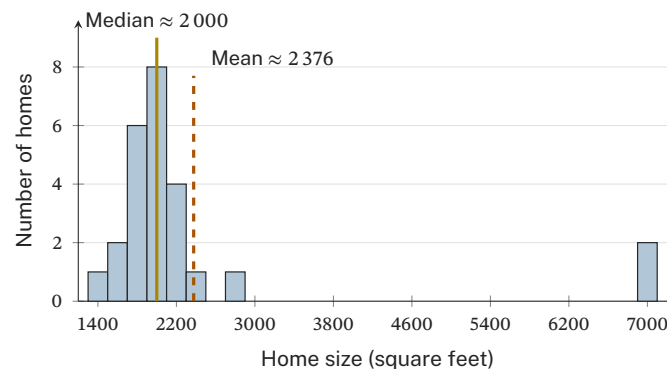


Figure 5. Histogram of home square footage with mean and median

The bar heights are reconstructed from the original Statology illustration: 1 400:1, 1 600:2, 1 800:6, 2 000:8, 2 200:4, 2 400:1, 2 800:1 and 7 000:2, for $n = 25$. Using bin midpoints, the median falls in the 2 000 bin and the mean is $59\,400/25 = 2\,376$. The median marker is therefore at 2 000. These are midpoint estimates from the displayed bins, not exact statistics recovered from unavailable individual home sizes.

5.3 Skewness and the order of the three statistics

Figure 6 shows how the ordering of the mean, median and mode shifts as the distribution's skew changes.

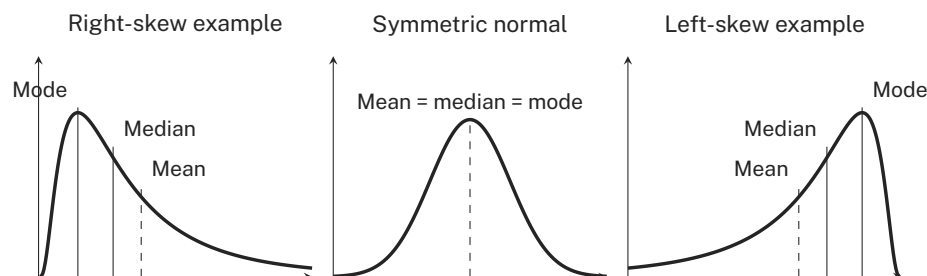


Figure 6. Mean, median and mode order under skew

A skewed distribution has a longer tail on one side. In these right-skewed examples, $\text{mode} < \text{median} < \text{mean}$, and the reflected left-skew example reverses it. This ordering is common but is not a theorem for every skewed distribution. The symmetric normal example has equal mean, median, and mode.

Interpretation guidance

Report the mean and the median together whenever the distribution may be skewed. A large gap between them is itself a finding. It says the distribution may have a long tail or contain values generated by a different mechanism, and it prompts a check of the plot and the measurement process. Choose the summary according to the question; the gap alone does not select the right statistic.

6 Q2. Statistics of variability, spread, or dispersion

6.1 Variance and standard deviation

$$s_x^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2, \quad s_x = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}.$$

The divisor $n - 1$ makes these the *sample* variance and sample standard deviation. Use this sample convention in Lab 01. Dividing the squared deviations by n instead describes variation across the complete set of observed values. The two conventions give different numbers; name the divisor when reporting variance or standard deviation. Repeated sensor readings can be related to one another, so these summaries alone do not justify claims about unobserved places or periods.

Discussion prompt: what are the units of variance? What about standard deviation? Variance is in the *squared* units of the variable, so if x is measured in dollars, s_x^2 is in dollars squared, which is less direct to interpret. The standard deviation is the square root, so it returns to the units of x and can be compared directly with the mean. That is why standard deviation is useful beside a measurement reported in its original units.

Worked example

For $x = 2, 4, 4, 5, 7, 9, 12, 41$ with $\bar{x} = 10.5$, the squared deviations are 72.25, 42.25, 42.25, 30.25, 12.25, 2.25, 2.25 and 930.25, summing to 1134. Therefore

$$s_x^2 = \frac{1134}{7} = 162, \quad s_x = \sqrt{162} = 12.73.$$

The single observation 41 contributes 930.25 of the 1134 total, which is 82% of the variability from one of eight observations.

R

```
var(x) # 162          divisor n - 1
sd(x)  # 12.72792

# Descriptive variance with divisor n
var(x) * (length(x) - 1) / length(x)
# 141.75
```

Python

```
x.var()          # 162.0      ddof = 1
x.std()          # 12.72792

x.var(ddof=0)    # 141.75, divisor n
x.std(ddof=1)    # 12.72792
```

R's `var` and `sd`, and pandas' defaults, divide by $n - 1$. In pandas, `ddof=1` means subtract one from the observed count in the divisor; `ddof=0` uses n . The sample standard deviation is 12.73, while the divisor- n version is 11.91.

6.2 Percentiles, quantiles, quartiles

The p th percentile locates the lower $p\%$ of a distribution. In samples, ties and interpolation mean that exactly $p\%$ need not lie below the reported value. Quartiles are the 25th, 50th, and 75th percentiles, denoted Q_1 , Q_2 , and Q_3 . The second quartile is the median, illustrated in Figure 7.

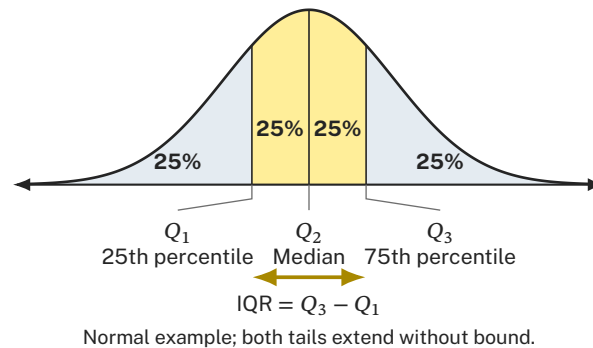


Figure 7. Quartiles and the interquartile range on a normal curve

Schematic after Wang [6]. The cut points enclose four equal probability regions. Unlike a finite sample, a normal distribution has no finite minimum or maximum. Sample 0th and 100th percentiles are the observed minimum and maximum, not the ends of this curve.

6.3 Which quantile definition did you get?

For Lab 01, use R's type-7 definition, which matches pandas' linear interpolation. Sort the n observed values, then locate $h = 1 + (n - 1)p$, counting positions from 1. If h lies between two positions, interpolate between their values. For Q_1 , $h = 2.75$ lies between two 4s, so $Q_1 = 4$. For Q_3 , $h = 6.25$ lies one quarter of the way from 9 to 12, giving $9 + 0.25(12 - 9) = 9.75$.

R

```
quantile(x, c(0.25, 0.75), type = 7)
# 25% 75%
# 4.00 9.75      default: type = 7

fivenum(x) # hinges, not type 7
# 2.0 4.0 6.0 10.5 41.0
```

Python

```
x.quantile([0.25, 0.75],
            interpolation="linear")
# 0.25 4.00
# 0.75 9.75    linear == R type 7
```

R's `fivenum` and default `boxplot` use Tukey's hinges, a different rule for dividing a sorted sample. Here the upper hinge is 10.5, while the type-7 upper quartile is 9.75. State which rule you use. The manual boxplot screening calculation in the practice below uses type 7 in both languages.

6.4 Interquartile range and range

Table 2 gives the interquartile range and range for the running example.

Table 2. Interquartile range and range for the running example

Statistic	Definition	Value for the running example
Interquartile range	$IQR = Q_3 - Q_1$	$9.75 - 4 = 5.75$
Range	maximum minus minimum	$41 - 2 = 39$

Discussion prompts: what is an advantage of using the IQR as a measure of spread? What is a disadvantage of using the range? The IQR describes the middle half of the data and ignores the

tails, so it is robust: moving the largest observation from 41 to 4,100 leaves it at 5.75. The range uses only the two most extreme observations, so it is determined entirely by the sample extremes, and it tends to increase with sample size under repeated sampling even when the underlying distribution does not change. In the running example the range is 39 and the IQR is 5.75, nearly a factor of seven apart. The largest observation strongly affects the range but not the middle-half spread.

R

```
IQR(x)      # 5.75
range(x)     # 2 41
diff(range(x)) # 39
```

Python

```
q1, q3 = x.quantile([0.25, 0.75],
                    interpolation="linear")
q3 - q1      # 5.75
(x.min(), x.max()) # (2, 41)
x.max() - x.min() # 39
```

Difference to know. R's `range()` returns the two endpoints, not their difference; `diff(range(x))` gives the spread. There is no IQR method in pandas, so the subtraction is written out, which has the side benefit of making the quantile definition visible.

7 Q3. Outliers and unknown values

7.1 What an outlier is

An outlier is an observation sufficiently unusual relative to the others to prompt investigation of its origin or measurement process.

A frequently used screening criterion is the **boxplot rule**. It flags observations outside the range

$$[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR].$$

Worked example

For $x = 2, 4, 4, 5, 7, 9, 12, 41$ we have $Q_1 = 4$, $Q_3 = 9.75$ and $IQR = 5.75$, so $1.5 \times IQR = 8.625$ and the interval is

$$[4 - 8.625, 9.75 + 8.625] = [-4.625, 18.375].$$

Only 41 lies outside it, so the rule flags exactly one value. The type-7 diagram below places whiskers at the most extreme observations inside these fences, 2 and 12. R's default boxplot uses hinges, giving $Q_3 = 10.5$ and fences -5.75 and 20.25 for this sample. Both approaches flag 41, but their boxes and numerical fences differ.

Figure 9 labels the same anatomy using a second illustrative scale.

Here $Q_1 = 7.72$, $Q_3 = 8.42$, and $IQR = 0.70$, so the fences are 6.67 and 9.47. Every separate circle lies outside those fences. The whiskers, at 6.82 and 9.10, stop inside them. These values define an illustrative diagram, not a recovered empirical dataset.

7.2 Missing and obviously wrong values

The discussion closes Q3 with two further categories of strange value: missing or unknown values, and other obviously wrong values, for example values of the wrong type.

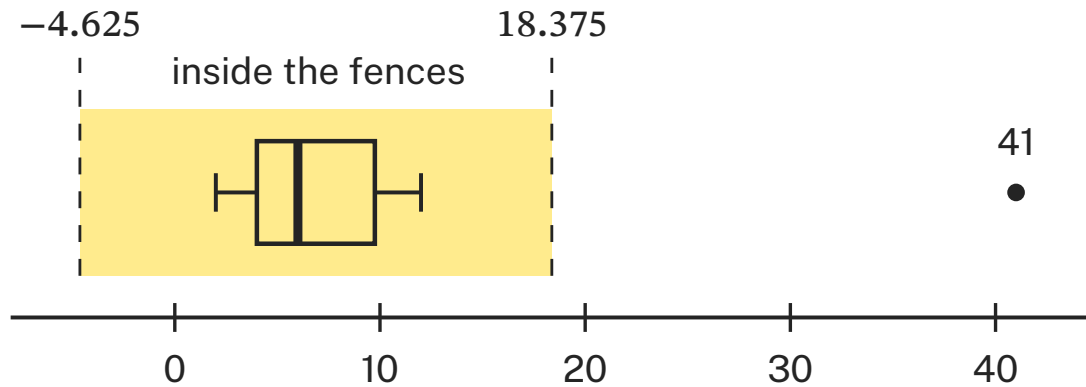


Figure 8. The running example, with type-7 quartiles, the two fences, observed whisker endpoints, and the single flagged value 41.

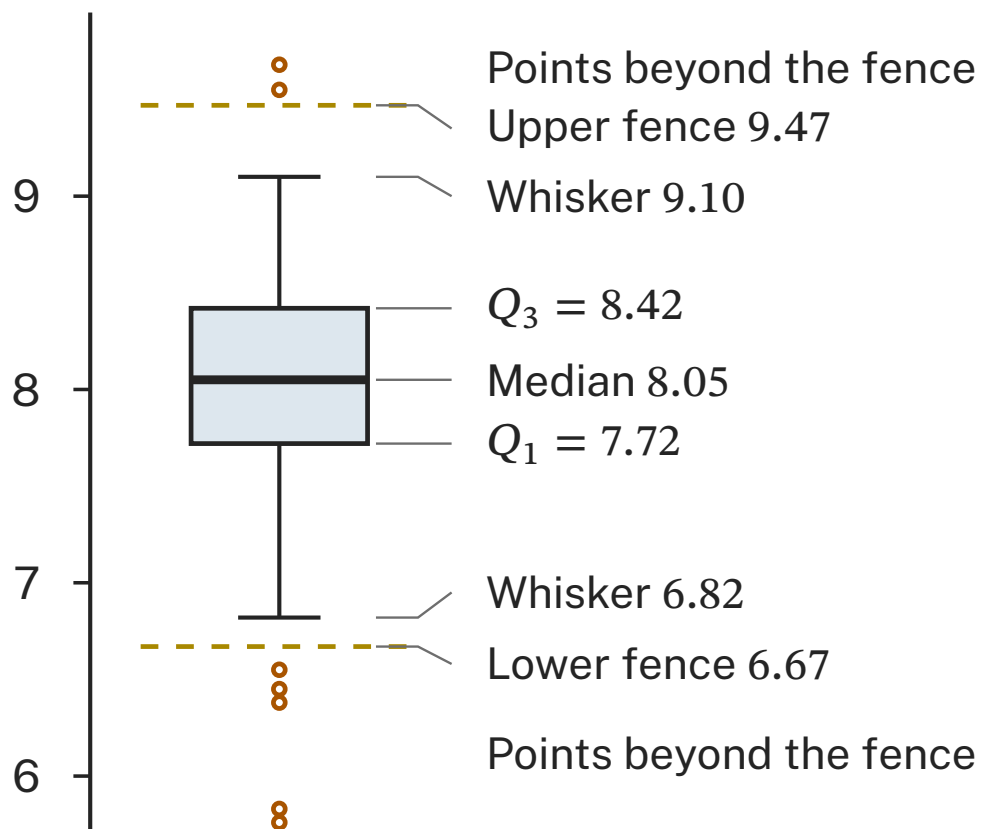


Figure 9. Anatomy of a box plot with labelled parts

R

```
x <- c(2,4,4,5,7,9,12,41,NA)

boxplot.stats(x)$out
# 41

which(is.na(x))
# 9

sum(is.na(x))
# 1
```

Python

```
x = pd.Series([2,4,4,5,7,9,12,41,None])

q1, q3 = x.quantile([0.25, 0.75],
                    interpolation="linear")

iqr = q3 - q1
x[(x < q1 - 1.5*iqr) | (x > q3 + 1.5*iqr)]
# 7    41.0

x.index[x.isna()]
# Index([8])
```

Difference to know. `boxplot.stats` builds its fences from Tukey's hinges rather than from quantile type 7, so the two approaches can disagree at the margin. Here both flag only 41.

Decision checkpoint

A flagged value is a question, not a verdict. Before changing or deleting a value, check the measurement process. An outlier can be a data-entry error, a unit mix-up, a legitimate rare event, or the single most interesting observation in the study. Record what you did and why, and report results with and without the flagged values when the conclusion changes.

8 Global summaries

Now combine the summaries already defined. R's `summary` and pandas' `describe` collect several of them in one call. Frequency tables count observations in each category.

R

```
x <- c(2, 4, 4, 5, 7, 9, 12, 41)

summary(x)
#   Min. 1st Qu.  Median    Mean
#   2.00   4.00   6.00   10.50
#   3rd Qu.    Max.
#   9.75   41.00

table(c("a", "b", "a", "c"))
# a b c
# 2 1 1
```

Python

```
import pandas as pd
x = pd.Series([2, 4, 4, 5, 7, 9, 12, 41])

x.describe()
# count      8.00    min      2.00
# mean     10.50    25%      4.00
# std      12.73    50%      6.00
#          75%      9.75
#          max     41.00

pd.Series(list("abac")).value_counts()
# a 2 / b 1 / c 1
```

Difference to know. R's `summary()` prints the mean but not the standard deviation; pandas' `describe()` prints both, and its `std` uses `ddof=1`, which matches R's `sd()`. The two outputs therefore look different while describing the same numbers.

9 Common misconceptions

- “The mean is the average, so it is always the right summary.” For a skewed variable, the median can describe the middle case more usefully. The mean remains relevant to questions about totals or expected values. Choose the summary to match the question.
- “The mode is the peak of a histogram.” The mode is the most frequent *value*. What a histogram shows is the most frequent *bin*, which depends on the bin width you chose.

- “*An outlier is an error.*” The boxplot rule is a screening device with no knowledge of the measurement process. A valid extreme measurement can be flagged. Check its units and source record before changing it.
- “*Quartiles are unambiguous.*” They are not; the worked example and R hinges illustrate two conventions. The differences can matter in small samples and near outlier thresholds. State the definition rather than assuming agreement.
- “*Standard deviation and variance say different things.*” They carry the same information; only the units differ. Report the standard deviation because it is interpretable next to the mean.

10 Discussion prompts from this class

1. What happens to the median in the hypothetical income example, and why does the mean behave so differently?
2. What are the units of variance? What about the standard deviation?
3. What do you think is an advantage of using the IQR as a measure of spread?
4. Can you think of a disadvantage of using the range?
5. Given a variable you have met in another course, which of the summaries in this class would you report, and which would you refuse to report?

11 Summary

Summarize data by calculating statistics of **location** (mean, median and mode) and statistics of **variability** (variance, standard deviation, interquartile range and range). Flag possible outliers using the box plot rule, then investigate values outside $[Q_1 - 1.5 \times IQR, Q_3 + 1.5 \times IQR]$.

Different statistics provide a different “picture” of the data and may be distorted by skewness of the distribution and by the presence of outliers. Report the choice of summary and its interpretation alongside the number.

The variable’s type decides which summaries exist at all; the software’s defaults decide which version of a summary you actually computed. Between R and Python the substantive differences to remember are the treatment of missing values, the ddof divisor for variance and standard deviation, and the sample-quantile definition behind any quartile.

12 In-class summary practice

The first hands-on practice runs in the scheduled Class B meeting. It covers getting acquainted with R or Python and generating basic data summaries, that is, computing by hand and in code every statistic defined in this class. Use one local or cloud environment and enter the running example during the warm-up. Check your output against $\bar{x} = 10.5$, median 6, mode 4, $s_x^2 = 162$, $s_x = 12.73$, $IQR = 5.75$ and range 39.

13 From a CSV to an analysis

The following small dataset is invented for classroom practice. Each row represents one sensor reading. The three measurements are temperature in degrees Celsius, oxygen saturation in percent, and

depth in metres. `record_id` is a stable row identifier and `site` defines two comparison groups. Neither is a measurement to average. This nine-row example cannot satisfy Lab 01's requirement for at least 100 observations from an approved source.

Create a folder for this practice and save the following text as `practice.csv`. CSV means comma-separated values. The first line gives column names; the blank temperature field on the last line records a missing measurement. A missing reading is different from a measured zero.

```
record_id,site,temperature_c,oxygen_percent,depth_m
r01,A,2,70,1
r02,A,4,80,2
r03,A,4,80,2
r04,A,5,85,3
r05,B,7,90,3
r06,B,9,95,4
r07,B,12,105,4
r08,B,41,130,10
r09,A,,75,2
```

Listing 1. The complete synthetic practice dataset.

13.1 Read the table and summarize its measurements

Choose one language. Place `practice.R` or `practice.py` beside the CSV, then assemble the following code blocks in order. Run R with `Rscript practice.R` or Python with `python practice.py` from that folder. In a cloud notebook, upload the CSV to the working folder and run the cells in order. Python needs `pandas` and `Matplotlib`; the R example uses functions included with R.

The loop visits each name in `numeric_cols`. Brackets select that column, and `observed` holds its non-missing values for calculation. The original table stays unchanged. Each iteration adds one row to the summary table, which is saved as a CSV in `results/`. The practice columns all have at least two observed values. In a real extract, check this before computing a sample standard deviation.

```
# Synthetic classroom data. Run from the folder containing practice.csv.
observations <- read.csv("practice.csv", stringsAsFactors = FALSE)
numeric_cols <- c("temperature_c", "oxygen_percent", "depth_m")
dir.create("results", showWarnings = FALSE)

summaries <- data.frame()
for (column in numeric_cols) {
  x <- observations[[column]]
  observed <- x[!is.na(x)]
  quartiles <- quantile(observed, c(0.25, 0.75), type = 7)
  row <- data.frame(
    variable = column, n = length(observed), missing = sum(is.na(x)),
    mean = mean(observed), median = median(observed),
    variance = var(observed), sd = sd(observed),
    q1 = unname(quartiles[1]), q3 = unname(quartiles[2]),
    iqr = unname(quartiles[2] - quartiles[1]),
    range = max(observed) - min(observed)
  )
  summaries <- rbind(summaries, row)
}
write.csv(summaries, "results/summaries.csv", row.names = FALSE)
```

Listing 2. R import and summaries.

```
# Synthetic classroom data. Run from the folder containing practice.csv.
from pathlib import Path

import matplotlib.pyplot as plt
import pandas as pd

observations = pd.read_csv("practice.csv")
numeric_cols = ["temperature_c", "oxygen_percent", "depth_m"]
Path("results").mkdir(exist_ok=True)

rows = []
for column in numeric_cols:
    x = observations[column]
    observed = x.dropna()
    q1, q3 = observed.quantile([0.25, 0.75], interpolation="linear")
    rows.append({
        "variable": column, "n": observed.size, "missing": x.isna().sum(),
        "mean": observed.mean(), "median": observed.median(),
        "variance": observed.var(ddof=1), "sd": observed.std(ddof=1),
        "q1": q1, "q3": q3, "iqr": q3 - q1,
        "range": observed.max() - observed.min(),
    })
pd.DataFrame(rows).to_csv("results/summaries.csv", index=False)
```

Listing 3. Python import and summaries.

For temperature, the output must show 8 observed values and 1 missing value, mean 10.5, median 6, sample variance 162, sample standard deviation about 12.73, $Q_1 = 4$, $Q_3 = 9.75$, IQR 5.75, and range 39. Check these against the hand calculations before adapting the code to an environmental extract.

13.2 Make one comparison

Define the groups from the question before screening unusual values. The next loop selects site A and site B in turn, then calculates their observed counts, missing counts, means, and medians. In R, the condition inside brackets selects rows. In pandas, `loc[condition, column]` selects rows and a column. Site A has 4 observed temperatures, 1 missing temperature, and mean 3.75; site B has 4 observed temperatures, no missing values, and mean 17.25. The value 41 contributes to that difference. These invented readings do not support a claim about actual sites or a cause of the difference.

```
groups <- data.frame()
for (site in c("A", "B")) {
  x <- observations$temperature_c[observations$site == site]
  row <- data.frame(
    site = site, n = sum(!is.na(x)), missing = sum(is.na(x)),
    mean = mean(x, na.rm = TRUE), median = median(x, na.rm = TRUE)
  )
  groups <- rbind(groups, row)
}
write.csv(groups, "results/groups.csv", row.names = FALSE)
```

Listing 4. R comparison.

```
rows = []
for site in ["A", "B"]:
    x = observations.loc[observations["site"] == site, "temperature_c"]
    rows.append({
        "site": site, "n": x.count(), "missing": x.isna().sum(),
        "mean": x.mean(), "median": x.median(),
    })
pd.DataFrame(rows).to_csv("results/groups.csv", index=False)
```

Listing 5. Python comparison.

13.3 Trace a flag back to its row

This example screens all observed temperatures together, using type-7 quartiles. The condition marks values below the lower fence or above the upper fence. The saved row retains its identifier and site, so you can trace the flag back to the input. Only r08, the reading of 41, is flagged. Screening within each site would ask a different question and can give a different result. For Lab 01, state which population of rows your fences describe and justify that choice.

```
x <- observations$temperature_c
quartiles <- quantile(x, c(0.25, 0.75), na.rm = TRUE, type = 7)
iqr <- quartiles[2] - quartiles[1]
flag <- !is.na(x) & (x < quartiles[1] - 1.5 * iqr |
                  x > quartiles[2] + 1.5 * iqr)
write.csv(observations[flag, ], "results/flagged.csv", row.names = FALSE)
```

Listing 6. R screening with type-7 quartiles.

```
x = observations["temperature_c"]
q1, q3 = x.quantile([0.25, 0.75], interpolation="linear")
iqr = q3 - q1
flag = x.notna() & ((x < q1 - 1.5 * iqr) | (x > q3 + 1.5 * iqr))
observations.loc[flag].to_csv("results/flagged.csv", index=False)
```

Listing 7. Python screening with linear interpolation.

For a real measurement, a statistical flag starts an investigation. Read the publisher's variable description and quality flags, check the units, and locate the original record. Retain a valid unusual value; correct or exclude a value only with documented evidence. The synthetic table has no publisher quality record, so it gives no evidence that 41 is a sensor error. In Lab 01, report the source check and your decision. If no values are flagged, inspect and describe the most extreme observations instead.

13.4 Save plots and document the run

The histogram uses the same ten-degree bins in both languages. It displays the eight observed temperatures and omits the missing reading. The scatter plot shows temperature against oxygen saturation for rows where both are observed. As in Class A, overlapping points can hide observations. Label every axis with the variable and its unit, and give the plotted count in your report. Association in a scatter plot does not establish cause and effect.

R's pdf opens a plot file and dev.off closes it. Matplotlib creates a figure and axes, saves the figure with savefig, and closes it with close. Each script also writes an environment record. Run the whole script again from a fresh session and confirm that it recreates the three tables, two plots, and environment file without manual edits.

```
pdf("results/temperature-histogram.pdf", width = 6, height = 4)
hist(x, breaks = c(0, 10, 20, 30, 40, 50), right = FALSE,
     main = "Practice temperature readings", xlab = "Temperature (C)",
     ylab = "Number of readings", col = "grey80")
dev.off()

pdf("results/temperature-oxygen.pdf", width = 6, height = 4)
plot(observations$temperature_c, observations$oxygen_percent,
     xlab = "Temperature (C)", ylab = "Oxygen saturation (%)", pch = 16)
dev.off()
writeLines(capture.output(sessionInfo()), "results/environment.txt")
```

Listing 8. R plots and environment record.

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(x.dropna(), bins=[0, 10, 20, 30, 40, 50], color="grey", edgecolor="black")
ax.set(title="Practice temperature readings", xlabel="Temperature (C)",
      ylabel="Number of readings")
fig.savefig("results/temperature-histogram.pdf", bbox_inches="tight")
plt.close(fig)

fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(observations["temperature_c"], observations["oxygen_percent"])
ax.set(xlabel="Temperature (C)", ylabel="Oxygen saturation (%)")
fig.savefig("results/temperature-oxygen.pdf", bbox_inches="tight")
plt.close(fig)
pd.show_versions(as_json="results/environment.json")
```

Listing 9. Python plots and environment record.

For Lab 01, keep the raw download unchanged and record its publisher, URL, licence, access date, filter choices, units, and missing-value codes in the README. A simple report can state the question, describe the extract, present one summary table and two plots, discuss checked unusual values, and state a limitation. A standard word processor exported to PDF is sufficient this week; automated reporting tools are introduced later.

14 Record and submit your work

A repository stores files and their revision history. A commit records a snapshot; its full SHA identifies that snapshot. A push sends local commits to GitHub. For Week 1, create a new **private** repository, give the instructor access in its collaborator settings, and keep it private until sharing is opened. Store Lab 01 under labs/lab-01/.

You can use GitHub's browser interface to create or upload files and choose *Commit changes*. Commit a meaningful stage of work each time, such as the imported table audit, the analysis, or the report. Before uploading, check that files contain only material allowed in the assessed repository. Keep the original download separate and document how to retrieve it.

If using local Git, first clone your private repository using the URL shown under its *Code* button. Open a terminal in that local repository. After editing the lab files, use the following sequence.

```
git remote -v
git status
git add labs/lab-01
git commit -m "Record Lab 01 analysis and evidence"
git push
git rev-parse HEAD
```

`git remote -v` shows the destination; confirm it is your private repository. `status` lists changed files, `add` selects the lab files, and `commit` records them. After a successful push, open that commit on GitHub and check its files. Copy the full SHA from the commit page or `git rev-parse HEAD`. Submit it and the commit URL through the announced private channel by the deadline. A branch link does not identify one fixed version.

Keep corrections as new commits. Do not amend, rebase, squash, delete, replace, force-push, or backdate assessment history. After the deadline and the instructor's release, publish the submitted commit with its history and present that version in class. Keep other unreleased work and private feedback out of the entire history being published. Follow syllabus A.6.1 if a private portfolio alternative is required.

GitHub repository setup Editing and committing in the browser

Lab 01 is individual integrated work, due September 12, 2026 at 23:59 Halifax time. The best 10 of 12 labs count for 15% of the course grade. Generative AI output is prohibited on integrated labs. The submission receipt and instructor-retained snapshot fix the assessed version. Released instructor materials may be shared with credit unless explicitly marked private; retain third-party attribution and licence terms.

15 Next lecture

The next class begins preprocessing foundations in *Data Preparation*. Later classes develop cleaning, transforming, feature engineering, and reducing dimensionality in the sequence specified by the syllabus. The highlighted stage shows where the next class begins.

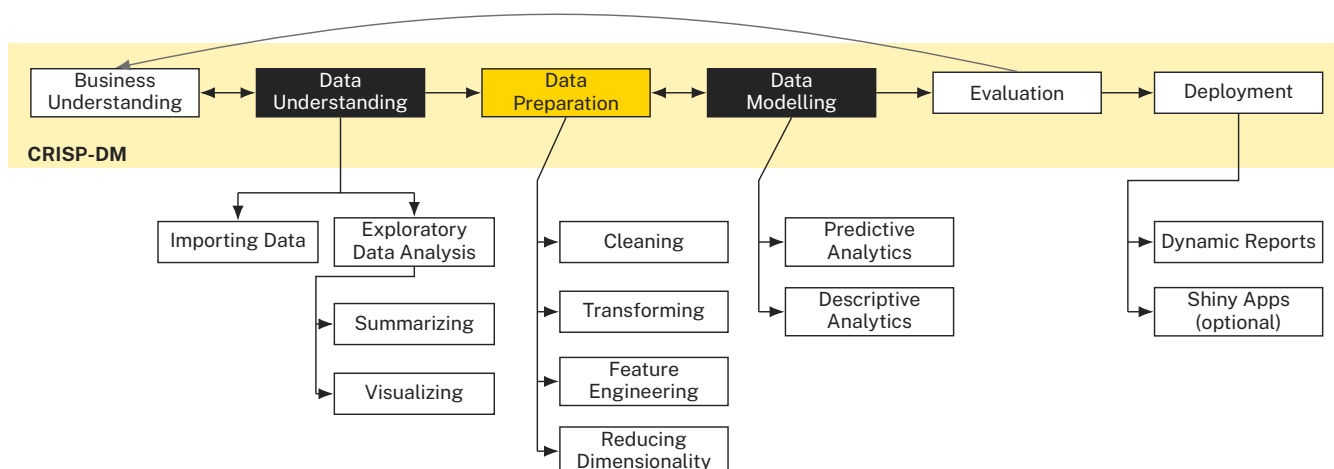


Figure 10. The same course map with data preparation highlighted for the next stage.

Integrated lab | Lab 01 | Atlantic environmental conditions

Choose one open Atlantic environmental dataset and establish the portfolio's descriptive baseline.

- **Carry forward.** Observational units, variable types, centre, spread, quantiles, missingness, and unusual-value investigation.
- **Start in class.** Choose one verified dataset, work through the selection checklist, and set up the R or Python analysis.
- **Assessment source.** week-01/Assessment/lab-01

16 Optional research examples

These examples extend Class A's idea of predicting a category. They are optional and are not needed for Lab 01. Training images are examples used to fit a classifier; test images are separate examples used to examine its predictions. A perturbation changes an input. Retraining fits the model again after its training data change. Confidence is the score assigned to a prediction, which can be high even for a mistake.



Figure 11. Background confounding. Left, illustrative tank photographs from Guidotti et al. [1]. Right, a husky classified as wolf and its explanation from Ribeiro et al., Figure 11 [4].

The tank story is apocryphal. Its photographs show different scenes and cannot establish the effect of changing only the background. The documented husky experiment is more specific. Ribeiro et al. intentionally selected training images that associated wolves with snow. Their explanation of a husky predicted as *wolf* retained mainly snow in the background. The second panel explains the same image; it does not show a different animal.



Figure 12. Adversarial test inputs from Szegedy et al., Figure 5 [5]. In each triplet the middle image magnifies the difference by 10x; the right-hand image is predicted as *ostrich*.

A test-input attack changes the image presented to an already trained model. Figure 12 separates the unchanged model from its perturbed input. The magnified difference is a visualization aid, not the image submitted to the classifier.

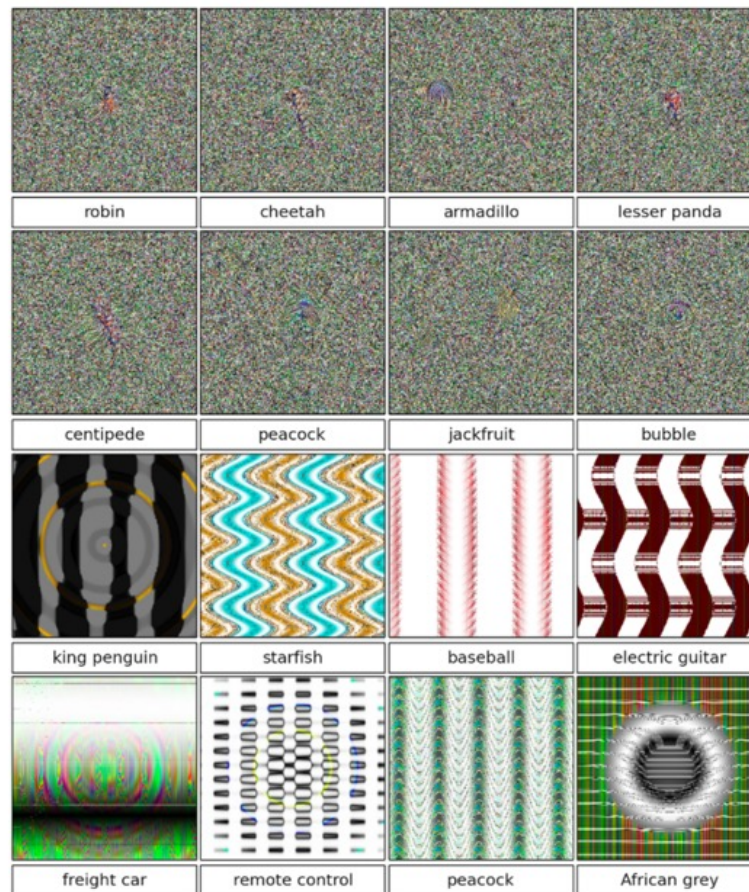


Figure 13. Optimized, unrecognizable images from Nguyen et al., Figure 1 [3]. Labels are network predictions. The source reports at least 99.6% confidence for these examples.

The noise-like and structured patterns in Figure 13 receive high-confidence class predictions despite not looking like those objects to a person. A model's confidence is not a guarantee that its input contains a recognizable object.

Training-data poisoning changes the data used to fit the model. In Figure 14, the training label stays *fish*; the ambiguous training image itself contains a dog and a fish. After perturbing that image and retraining, predictions flip from dog to fish for 16 of 30 targeted test images. The test photographs are not transformed into fish. This separates training-time corruption from a perturbation applied only at prediction time. The cited papers are optional follow-up reading.

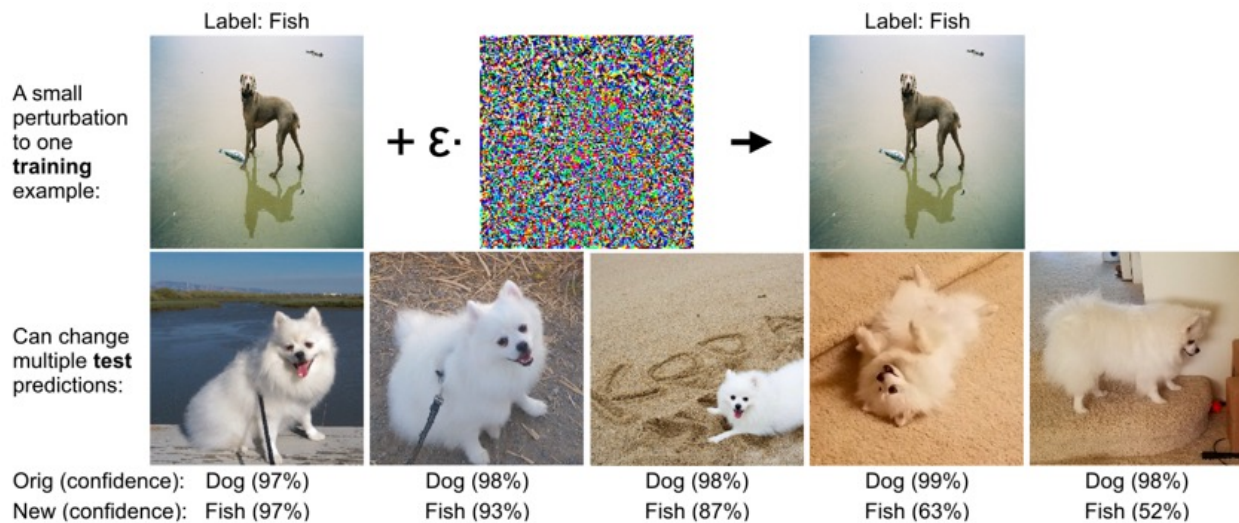


Figure 14. Training-data poisoning from Koh and Liang, Figure 5 [2]. The top row changes one training image without changing its label. The bottom row shows separate, unchanged test images.

References

- [1] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi. “A survey of methods for explaining black box models”. In: *ACM Computing Surveys* 51.5 (2018), pp. 1–42.
- [2] P. W. Koh and P. Liang. “Understanding black-box predictions via influence functions”. In: *Proceedings of the 34th International Conference on Machine Learning*. PMLR, 2017, pp. 1885–1894.
- [3] A. Nguyen, J. Yosinski, and J. Clune. “Deep neural networks are easily fooled: high confidence predictions for unrecognizable images”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2015, pp. 427–436.
- [4] M. T. Ribeiro, S. Singh, and C. Guestrin. ““Why should I trust you?” Explaining the predictions of any classifier”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 1135–1144.
- [5] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. *Intriguing properties of neural networks*. arXiv preprint arXiv:1312.6199. 2013.
- [6] S. Wang. “An adaptive ensemble classification framework for real-time data streams by distributed control systems”. In: *Neural Computing and Applications* 32.9 (2020), pp. 4139–4149.